Retrospectiva de la Versión uno

DOPOSoccer 2026-2 · Desarrollo Orientado por Objetos · Escuela Colombiana de Ingeniería

1. Tiempo invertido

Juan Diego Castaño Parra: 18 horas.

Juan Diego Carreño Gutierrez: 17 horas.

Total: 35 horas.

2. Estado actual del proyecto

La Versión uno está **funcional, con fallas conocidas**. Dos jugadores pueden configurar y jugar un partido completo en el mismo teclado. Priorizamos lo necesario para jugar de principio a fin; por eso quedan funciones para las siguientes versiones y fallas por corregir (sección 6).

Qué incluye

- **Configuración:** tres resoluciones (800×450, 1024×576 y 1280×720), partido por goles o por tiempo, y cantidad de goles o minutos.
- **Juego:** movimiento, salto, patada, daño y aturdimiento; balón con gravedad, rebotes y choques; goles y nuevo saque.
- **Información en pantalla:** marcador, tiempo o meta de goles, vida y barra de poder de cada jugador.
- **Control del partido:** pausa, terminar en cualquier momento, y mensaje final con ganador y puntos.
- **Pruebas:** 140 pruebas unitarias del dominio, todas pasan.

Qué no incluye todavía

Activación de poderes y aura, personajes, jugador máquina, guardar y abrir partidas, y mejora de estadísticas con puntos. La barra de poder se carga y se muestra, pero aún no se puede usar.

3. Mayor logro

Tener el juego completo funcionando. Se configura, se juega y termina con ganador y puntos, con el dominio y la presentación conectados a través de la clase `DOPOSoccer`. Es el logro más importante porque el proyecto ya se usa de punta a punta y no es un conjunto de piezas sueltas.

4. Mayor problema

El poco tiempo entre la subida de los requisitos finales y el límite de la entrega. Para resolverlo:

- Priorizamos el nivel básico Jugador vs Jugador y dejamos el resto para las siguientes versiones.
- Repartimos el trabajo por capas, dominio y presentación, para avanzar en paralelo.
- Acordamos la fachada `DOPOSoccer` como contrato entre las dos capas.
- Escribimos pruebas unitarias del dominio para cambiar el código sin repetir toda la revisión a mano.
- Integramos con ramas y Pull Requests para no pisarnos el trabajo.

5. Trabajo en equipo

Qué hicimos bien: tuvimos un buen reparto y buena coordinación. Dividimos el trabajo entre dominio y presentación, acordamos cómo se conectaban las capas y unimos los cambios por ramas.

A qué nos comprometemos: mantener las ramas y los Pull Requests, y planear desde que se publiquen los requisitos para dejar más tiempo de integración y prueba conjunta.

6. Fallas conocidas y deuda técnica

Encontradas al revisar la rama `develop`:

- **Arcos:** el balón puede entrar por encima del arco y contar como gol. El enunciado pide que rebote en los arcos.
- **Pausa:** no bloquea saltar ni patear. La patada se activa enseguida y el salto arranca al reanudar.
- **Tecla tras un gol:** un jugador que sigue presionando una tecla puede quedarse quieto hasta soltarla y volver a presionarla.
- **Patada en pantalla:** la zona real mide 60×50 y el pie dibujado 40×20, separado del cuerpo. El balón puede salir golpeado sin tocar el pie dibujado.
- **Información en pantalla:** no se muestran los controles, y las barras no indican de quién son ni cuál es vida o poder.
- **Ventana:** se puede redimensionar y deforma la cancha. Si pierde el foco con una tecla presionada, el jugador puede seguir caminando.
- **Código:** `esPorTiempo()` y `getSegundosRestantes()` ya no se usan en la interfaz; `Controles` no tiene pruebas unitarias; el JUnit de `lib/` no está en el repositorio.

7. Referencias y herramientas

- **Referencia:** enunciado del proyecto DOPOSoccer 2026-2.
- **Herramientas:** Java 21 con Swing, JUnit 6, Visual Studio Code, Astah UML, Git y GitHub.
- **Inteligencia artificial:** usamos Claude como apoyo en la guía de implementación, la revisión de código, las pruebas y la redacción de este documento.